



Hand Cricket Game

Full-Stack Code Walkthrough — Every File, Every Piece of Logic Explained

React · Node.js · Express · MongoDB · Markov-Model AI

Generated for Vivek Mishra · Project path: hand-cricket-game/

Table of Contents

1. How to Read This Document
2. Backend — server.js (entry point)
3. Backend — config/database.js
4. Backend — models/User.js
5. Backend — models/Game.js
6. Backend — models/Leaderboard.js
7. Backend — middleware/auth.js
8. Backend — utils/gameLogic.js
9. Backend — utils/markovModel.js (the AI)
10. Backend — routes/auth.js
11. Backend — routes/game.js (the core game engine)
12. Backend — routes/leaderboard.js
13. Frontend — main.jsx (entry point)
14. Frontend — App.jsx (routing)
15. Frontend — context/AuthContext.jsx
16. Frontend — api/api.js
17. Frontend — components/Auth/Login.jsx & Signup.jsx
18. Frontend — components/Game/Game.jsx
19. Frontend — components/Game/GameBoard.jsx (gameplay engine)
20. Frontend — components/Leaderboard & Profile
21. End-to-End Request Lifecycle

1. How to Read This Document

Every file section below has three parts:

- **Purpose box** — what the file is responsible for, in one or two sentences.
- **Full code** — the actual source, exactly as it exists in the project.
- **Line-by-line / block-by-block table** — the code snippet on the left, plain-English explanation of the syntax and the reasoning behind it on the right.

The document is ordered the way the app actually executes: backend entry point first, then every backend file it depends on, then the frontend entry point, then every frontend file in the order a user encounters them (signup → login → play a game → view stats).

2. server/server.js — Backend Entry Point

server/server.js

This is the very first file Node.js executes when you run `npm run dev` in the `server/` folder. It creates the Express app, connects to MongoDB, wires up every route file, and starts the HTTP server listening on port 5000.

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const http = require('http');
const socketIo = require('socket.io');
const connectDB = require('./config/database');

const authRoutes = require('./routes/auth');
const gameRoutes = require('./routes/game');
const leaderboardRoutes = require('./routes/leaderboard');

const app = express();
const server = http.createServer(app);
const io = socketIo(server, {
  cors: {
    origin: process.env.CLIENT_URL || 'http://localhost:3000',
    methods: ['GET', 'POST'],
  },
});

connectDB();

app.use(express.json());
app.use(cors());

app.use('/api/auth', authRoutes);
app.use('/api/game', gameRoutes);
app.use('/api/leaderboard', leaderboardRoutes);

app.get('/health', (req, res) => {
  res.json({ status: 'Server is running' });
});

const connectedUsers = new Map();

io.on('connection', (socket) => {
  socket.on('user-join', (userId) => {
    connectedUsers.set(userId, socket.id);
    io.emit('user-online', { userId, count: connectedUsers.size });
  });
  socket.on('disconnect', () => {
    connectedUsers.forEach((value, key) => {
      if (value === socket.id) connectedUsers.delete(key);
    });
  });
});

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Code	Explanation
<code>require('dotenv').config();</code>	Loads key/value pairs from the <code>.env</code> file into <code>process.env</code> . Must run before any code that reads <code>process.env.MONGODB_URI</code> , <code>JWT_SECRET</code> , etc. — that's why it's the very first line in the file.
<code>const app = express();</code>	Creates an Express application instance — the object you attach routes and middleware to.
<code>const server = http.createServer(app);</code>	Express's app alone can serve HTTP, but Socket.IO needs a raw Node <code>http.Server</code> to attach to. Wrapping app in <code>http.createServer</code> gives one object that does both.
<code>const io = socketIo(server, {cors:{...}})</code>	Attaches Socket.IO to that server, with a CORS allow-list so the React dev server (a different origin/port) is permitted to open a WebSocket connection.
<code>connectDB();</code>	Fires off the MongoDB connection (see file #3). Not awaited here — Mongoose queues operations until the connection is ready, so it's safe to keep booting the server in parallel.
<code>app.use(express.json());</code>	Built-in Express middleware. Without it, <code>req.body</code> would be undefined for JSON POST requests — this parses the raw request body into a JS object.
<code>app.use(cors());</code>	Adds Access-Control-Allow-Origin headers to every response so the browser doesn't block cross-origin requests from the React app (<code>localhost:5173</code>) to the API (<code>localhost:5000</code>).
<code>app.use('/api/auth', authRoutes);</code>	Mounts an entire router (defined in <code>routes/auth.js</code>) under a URL prefix. Any route defined as <code>/login</code> inside that file becomes <code>/api/auth/login</code> here.
<code>io.on('connection', (socket) => {...})</code>	Fires once per browser tab that opens a WebSocket to this server. <code>socket.on('user-join', ...)</code> registers a listener for a custom event the frontend could emit (infrastructure for future real-time features — the game itself currently runs over REST, not sockets).
<code>server.listen(PORT, () => {...})</code>	Starts the server actually listening for incoming connections. The callback runs once the OS confirms the port is bound.

3. server/config/database.js — MongoDB Connection

server/config/database.js

A small, isolated module whose only job is opening (and reporting on) the MongoDB connection, so `server.js` doesn't need to know connection details.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
    return conn;
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Code	Explanation
<code>async () => { ... await mongoose.connect(...) }</code>	<code>mongoose.connect</code> returns a Promise. <code>async/await</code> lets the code read top-to-bottom instead of chaining <code>.then()</code> calls. Any rejection (bad URI, MongoDB not running) is caught below rather than crashing with an unhandled rejection.
<code>process.exit(1);</code>	If the database is unreachable, the app is useless — this deliberately kills the Node process with a non-zero exit code (signals failure to any process manager watching it) rather than limping along with a broken DB layer.
<code>module.exports = connectDB;</code>	CommonJS export — makes this function importable elsewhere via <code>require('./config/database')</code> .

Earlier version of this file passed `{ useNewUrlParser: true, useUnifiedTopology: true }` as a second argument — those flags were removed because the installed Mongoose driver version no longer accepts them; they're on by default now. Passing them caused a hard crash on every connection attempt.

4. server/models/User.js — User Schema

server/models/User.js

Defines the shape of a "user" document in MongoDB, plus two pieces of behavior attached directly to the schema: automatic password hashing, and a method to check a password at login time.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  profilePicture: { type: String, default: null },
  stats: {
    gamesPlayed: { type: Number, default: 0 },
    gamesWon: { type: Number, default: 0 },
    totalRuns: { type: Number, default: 0 },
    wickets: { type: Number, default: 0 },
  },
  createdAt: { type: Date, default: Date.now },
});

userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

userSchema.methods.comparePassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Code	Explanation
<code>new mongoose.Schema({...})</code>	Declares field names, types, and validation rules. Mongoose enforces these at the application layer before anything reaches MongoDB (which itself is schemaless).
<code>unique: true (on email)</code>	Tells Mongoose to create a unique index on this field in MongoDB — a duplicate email insert will throw a database-level error, which the signup route catches and turns into a friendly "Email already registered" message.
<code>stats: { gamesPlayed: {...}, ... }</code>	A nested (embedded) sub-document. Because a user's stats always belong to exactly one user and are always read together, embedding avoids a second collection + join/populate for something this simple.
<code>userSchema.pre('save', async function () {...})</code>	A Mongoose "hook" — this function runs automatically every time <code>.save()</code> is called on a User document, <i>before</i> it's written to the DB. Using a regular function (not an arrow function) is required here so that <code>this</code> refers to the document being saved.
<code>if (!this.isModified('password')) return;</code>	Guards against re-hashing an already-hashed password. Without this, updating a user's <i>name</i> would re-hash the (already hashed) password on every save, corrupting it.
<code>bcrypt.genSalt(10) / bcrypt.hash(...)</code>	Generates a random salt (10 = the hashing "cost factor" — higher is slower but more resistant to brute force) and combines it with the plaintext password to produce a one-way hash. The plaintext password is never stored.
<code>userSchema.methods.comparePassword = ...</code>	Adds a custom instance method to every User document. <code>bcrypt.compare</code> re-hashes the entered password with the same stored salt and checks if it matches — this is the only way to "check" a bcrypt hash, since hashing isn't reversible.
<code>mongoose.model('User', userSchema)</code>	Compiles the schema into an actual model class with <code>.find()</code> , <code>.save()</code> , etc. The string 'User' becomes the MongoDB collection name (pluralized/lowercased to users).

5. server/models/Game.js — Game State Schema

server/models/Game.js

Represents one full match from toss to finish. This is the most complex schema because it holds live, mutable state that gets updated ball-by-ball.

```
const gameSchema = new mongoose.Schema({
  gameId: { type: String, required: true, unique: true },
  player1: {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    name: String,
    lastNumber: { type: Number, min: 1, max: 6, default: null },
    numberHistory: { type: [Number], default: [] },
    runs: { type: Number, default: 0 },
    wickets: { type: Number, default: 0 },
  },
  player2: {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: String,
    difficulty: { type: String, enum: ['easy', 'medium', 'hard'], default: 'medium' },
    lastNumber: { type: Number, min: 1, max: 6, default: null },
    runs: { type: Number, default: 0 },
    wickets: { type: Number, default: 0 },
    isBot: { type: Boolean, default: false },
  },
  status: {
    type: String,
    enum: ['waiting', 'toss', 'toss-choice', 'inning1', 'inning2', 'finished'],
    default: 'waiting',
  },
  currentInning: { type: Number, default: 1 },
  target: { type: Number, default: null },
  tossWinner: String,
  batting: String,
  bowling: String,
  winner: String,
  winType: String,
  createdAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model('Game', gameSchema);
```

Code	Explanation
<code>userId: { type: ObjectId, ref: 'User' }</code>	<code>ref: 'User'</code> is what makes <code>.populate('player1.userId')</code> work later (used in the GET <code>/game/:id</code> route) — it tells Mongoose which collection to look up when resolving that <code>ObjectId</code> into a full document.
<code>numberHistory: { type: [Number], default: [] }</code>	An array field. Every number the human player picks during a match gets pushed here. This is the raw data the Markov-model AI (file #9) analyzes to predict the player's next move.
<code>difficulty: { enum: ['easy', 'medium', 'hard'] }</code>	Restricts the field to exactly these three string values — MongoDB will reject a save with anything else, which is a validation safety net if a bad request ever slips past the route handler's own checks.
<code>status: { enum: [...], default: 'waiting' }</code>	This single field drives the entire game state machine. <code>toss-choice</code> only appears when the <i>human</i> wins the toss and needs to pick bat/bowl; if the bot wins, the game skips straight from <code>toss</code> to <code>inning1</code> .
<code>batting / bowling: String</code>	Store either <code>'player1'</code> or <code>'player2'</code> — used throughout <code>routes/game.js</code> as a dynamic key, e.g. <code>game[battingKey].runs += ...</code> , so the exact same code path handles both innings regardless of who's currently batting.
<code>target: { type: Number, default: null }</code>	Only set once <code>inning 1</code> ends — becomes "runs needed to win" for whoever bats second. Stays <code>null</code> throughout <code>inning 1</code> .

6. server/models/Leaderboard.js — Ranking Table

server/models/Leaderboard.js

A denormalized summary — one document per user — that's cheap to sort and query for the leaderboard screen, instead of recalculating win rates from every Game document on every page load.

```
const leaderboardSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, unique: true },
  userName: String,
  winRate: { type: Number, default: 0 },
  totalGames: { type: Number, default: 0 },
  totalWins: { type: Number, default: 0 },
  totalRuns: { type: Number, default: 0 },
  totalWickets: { type: Number, default: 0 },
  avgRunsPerGame: { type: Number, default: 0 },
  rank: { type: Number, default: 0 },
  updatedAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model('Leaderboard', leaderboardSchema);
```

Code	Explanation
<code>userId: { ..., unique: true }</code>	Enforces exactly one leaderboard row per user. Combined with <code>findOneAndUpdate(..., {upsert:true})</code> in the game routes, this pattern means "update my row if it exists, create it if it doesn't" without a separate exists-check.
<code>rank: { type: Number, default: 0 }</code>	Stored but actually recalculated on read (see <code>routes/leaderboard.js</code>) by counting how many users have a higher <code>winRate</code> — this field exists mainly as a convenience/cache slot.

7. server/middleware/auth.js — JWT Gatekeeper

server/middleware/auth.js

Express "middleware" is a function that runs before a route handler and can either let the request continue (`next()`) or short-circuit it. This one verifies the JWT on protected routes.

```
const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  try {
    const token = req.header('Authorization')?.replace('Bearer ', '');
    if (!token) {
      return res.status(401).json({ message: 'No authentication token, access denied' });
    }
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    res.status(401).json({ message: 'Token is not valid' });
  }
};

module.exports = auth;
```

Code	Explanation
<code>req.header('Authorization')?.replace('Bearer ', '')</code>	Clients send the token as <code>Authorization: Bearer <token></code> . The optional-chaining <code>?.</code> avoids a crash if the header is missing entirely (returns <code>undefined</code> instead of throwing).
<code>jwt.verify(token, process.env.JWT_SECRET)</code>	Cryptographically checks the token's signature against the server's secret key, and checks it hasn't expired. Throws if either check fails — caught by the surrounding <code>try/catch</code> .
<code>req.userId = decoded.userId; next();</code>	Attaches the authenticated user's id onto the request object so every downstream route handler can read <code>req.userId</code> without re-verifying anything. <code>next()</code> is the Express signal to proceed to the actual route handler.
<code>router.post('/create', auth, async (req,res)=>{...})</code>	(usage elsewhere) Passing <code>auth</code> as a second argument means Express runs it first; if it calls <code>next()</code> , the third argument (the real handler) runs next. If <code>auth</code> instead sends a response (401), the real handler never executes.

8. server/utils/gameLogic.js — Pure Cricket Rules

server/utils/gameLogic.js

Deliberately framework-free — no Express, no Mongoose, no req/res. Just the mathematical rules of hand cricket, kept as static methods so they're trivial to reason about and unit-test in isolation.

```
class GameLogic {
  static playBall(batsmanNumber, bowlerNumber) {
    if (batsmanNumber === bowlerNumber) {
      return { outcome: 'wicket', runs: 0 };
    }
    return { outcome: 'runs', runs: batsmanNumber };
  }

  static resolveToss(guess, number1, number2) {
    const sum = number1 + number2;
    const parity = sum % 2 === 0 ? 'even' : 'odd';
    const winner = parity === guess ? 'player1' : 'player2';
    return { sum, parity, winner };
  }

  static calculateWinRate(gamesWon, gamesPlayed) {
    if (gamesPlayed === 0) return 0;
    return ((gamesWon / gamesPlayed) * 100).toFixed(2);
  }

  static generateGameId() {
    return `game_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
  }

  static isValidNumber(number) {
    return Number.isInteger(number) && number >= 1 && number <= 6;
  }
}

module.exports = GameLogic;
```

Code	Explanation
<pre>static playBall(batsmanNumber, bowlerNumber)</pre>	The entire rule of hand cricket in one comparison: if both players show the same number, that's a dismissal (a "wicket"); otherwise the batsman scores runs exactly equal to the number they showed. <code>static</code> means you call it as <code>GameLogic.playBall(...)</code> without needing to instantiate the class.
<pre>const parity = sum % 2 === 0 ? 'even' : 'odd';</pre>	The modulo operator <code>%</code> gives the remainder of division by 2 — 0 means even. This is the real-world "odd or even" hand-cricket toss rule: whoever correctly guesses the parity of the two thrown numbers' sum wins the toss.
<pre>const winner = parity === guess ? 'player1' : 'player2';</pre>	The human always makes the guess; if it matches the actual parity, <code>player1</code> (the human) wins the toss, otherwise the bot (<code>player2</code>) does. Notice a tie is mathematically impossible here — a sum is always definitively odd or even, unlike rock-paper-scissors which can tie.
<pre>Date.now() + Math.random().toString(36)...</pre>	Builds a unique-enough game ID by combining the current timestamp (always increasing) with a random base-36 string, without needing a database round-trip or an external UUID library.
<pre>Number.isInteger(number) && number >= 1 && number <= 6</pre>	Input validation used by every route that accepts a player's hand-cricket number — rejects decimals, strings that coerced to numbers, or out-of-range values before they ever touch game logic.

9. server/utils/markovModel.js — The AI Opponent

server/utils/markovModel.js

Implements the bot's "intelligence": a Markov chain that learns which number the human tends to play next, based on what they just played, then decides whether to match it (bowling, to get a wicket) or avoid it (batting, to keep scoring).

```

const NUMBERS = [1, 2, 3, 4, 5, 6];

class MarkovModel {
  constructor(difficulty = 'medium') {
    this.difficulty = difficulty;
    if (difficulty === 'easy') this.randomness = 0.7;
    else if (difficulty === 'hard') this.randomness = 0.15;
    else this.randomness = 0.4;
  }

  getRandomNumber() {
    return NUMBERS[Math.floor(Math.random() * NUMBERS.length)];
  }

  buildTransitionCounts(history) {
    const counts = {};
    NUMBERS.forEach((n) => {
      counts[n] = {};
      NUMBERS.forEach((next) => { counts[n][next] = 1; });
    });
    for (let i = 0; i < history.length - 1; i++) {
      const current = history[i];
      const next = history[i + 1];
      counts[current][next] += 1;
    }
    return counts;
  }

  predictNextNumber(history) {
    if (!history || history.length === 0) return this.getRandomNumber();
    if (Math.random() < this.randomness) return this.getRandomNumber();

    const lastNumber = history[history.length - 1];
    const counts = this.buildTransitionCounts(history);
    const distribution = counts[lastNumber];
    const total = Object.values(distribution).reduce((sum, c) => sum + c, 0);

    let predicted = lastNumber;
    let highestProb = -1;
    NUMBERS.forEach((n) => {
      const prob = distribution[n] / total;
      if (prob > highestProb) { highestProb = prob; predicted = n; }
    });
    return predicted;
  }

  chooseBotNumber(opponentHistory, botRole) {
    if (Math.random() < this.randomness) return this.getRandomNumber();
    const predictedOpponentNumber = this.predictNextNumber(opponentHistory);
    if (botRole === 'bowl') return predictedOpponentNumber;

    let avoidNumber = this.getRandomNumber();
    while (avoidNumber === predictedOpponentNumber) {
      avoidNumber = this.getRandomNumber();
    }
    return avoidNumber;
  }
}

```

```

}

module.exports = MarkovModel;

```

Code	Explanation
<code>this.randomness = 0.7 / 0.4 / 0.15</code>	The single knob that controls difficulty. It's the probability the bot ignores its own prediction and just picks randomly. Easy bots are "distracted" 70% of the time; hard bots only 15% of the time — meaning hard mode leans on the learned pattern much more aggressively.
<code>buildTransitionCounts(history)</code>	Builds a 6×6 "transition matrix": for every number 1–6, how many times has each other number 1–6 immediately followed it in the player's history? This is a textbook first-order Markov chain — the prediction only looks at the <i>immediately previous</i> number, not the whole history at once.
<code>counts[n][next] = 1;</code>	Every cell starts at 1, not 0 — this is Laplace/"add-one" smoothing. Without it, a number combination that's never occurred yet would have a computed probability of exactly 0 and could never be predicted, even after just one data point skews things.
<code>for (let i = 0; i < history.length - 1; i++)</code>	Walks the history in overlapping pairs — (history[0]→history[1]), (history[1]→history[2]), etc. — incrementing the count each time that specific transition is observed.
<code>predictNextNumber(history)</code>	Looks at the last number the player showed, finds which number most often follows it historically (highest count ÷ total = highest probability), and returns that as the prediction.
<code>chooseBotNumber(opponentHistory, botRole)</code>	The bot's actual decision function, used both when bowling and batting. If bowling, it wants to <i>match</i> the batsman's predicted number (get a wicket). If batting, it deliberately loops until it picks something <i>different</i> from the predicted bowler number, to avoid getting out.
<code>while (avoidNumber === predictedOpponentNumber) { ... }</code>	A retry loop — keeps re-rolling a random number until it lands on something other than the predicted one. With 6 possible numbers this terminates almost immediately on average.

10. server/routes/auth.js — Signup / Login / Me

server/routes/auth.js

The three endpoints that manage account creation, authentication, and fetching the currently logged-in user's profile.

```
const generateToken = (userId) => {
  return jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '7d' });
};

router.post('/signup', async (req, res) => {
  const { name, email, password, confirmPassword } = req.body;
  if (!name || !email || !password || !confirmPassword) {
    return res.status(400).json({ message: 'All fields are required' });
  }
  if (password !== confirmPassword) {
    return res.status(400).json({ message: 'Passwords do not match' });
  }
  const existingUser = await User.findOne({ email });
  if (existingUser) {
    return res.status(400).json({ message: 'Email already registered' });
  }
  const user = new User({ name, email, password });
  await user.save();
  const token = generateToken(user._id);
  res.status(201).json({ message: 'User created successfully', token, user: {...} });
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ message: 'Invalid email or password' });
  const isValid = await user.comparePassword(password);
  if (!isValid) return res.status(401).json({ message: 'Invalid email or password' });
  const token = generateToken(user._id);
  res.json({ message: 'Login successful', token, user: {...} });
});

router.get('/me', auth, async (req, res) => {
  const user = await User.findById(req.userId).select('-password');
  res.json({ user: {...} });
});
```

Code	Explanation
<code>jwt.sign({ userId }, SECRET, { expiresIn: '7d' })</code>	Creates a signed token encoding the user's id, valid for 7 days. Anyone with the server's JWT_SECRET can verify it's authentic; nobody without it can forge one.
<code>if (!name !email !password !confirmPassword)</code>	Falsy-check shorthand — catches missing fields, empty strings, null, and undefined all in one condition, before hitting the database at all.
<code>const existingUser = await User.findOne({ email });</code>	An explicit pre-check for a friendlier error message. (The schema's <code>unique: true</code> index would also reject a duplicate at the database level, but that error is uglier to parse and handle in the response.)
<code>const user = new User({...}); await user.save();</code>	Constructing the model doesn't touch the database — <code>.save()</code> is what actually inserts it, and is also what triggers the <code>pre('save')</code> password-hashing hook from <code>models/User.js</code> .
<code>res.status(201).json({...})</code>	HTTP 201 = "Created" — the correct status code for a successful POST that creates a new resource (as opposed to 200 for a plain success).
<code>await user.comparePassword(password)</code>	Calls the instance method defined on the schema (file #4) — internally runs <code>bcrypt.compare</code> against the stored hash. Returns a boolean.
<code>.select('-password')</code>	Mongoose query modifier — the leading minus sign means "exclude this field." Even though the password is hashed, there's no reason to ever send it back to the client.
<code>router.get('/me', auth, async (req,res)=> {...})</code>	This route requires a valid JWT (the auth middleware runs first) — it's how the frontend re-establishes "who am I logged in as" after a page refresh, using only the token in <code>localStorage</code> .

11. server/routes/game.js — The Core Game Engine

server/routes/game.js

The largest and most important backend file. Five endpoints drive a match end-to-end: create → toss → (optional) choose bat/bowl → repeated play → finish. A shared `finalizeGame()` helper is used by two different endpoints to avoid duplicating the "close out a match" logic.

11a. finalizeGame(game) — shared helper

```
async function finalizeGame(game) {
  const player1Runs = game.player1.runs;
  const player2Runs = game.player2.runs;
  let winner = 'tie';
  let winType = 'tie';

  if (player1Runs !== player2Runs) {
    winner = player1Runs > player2Runs ? 'player1' : 'player2';
    const chasingKey = game.currentInning === 2 ? game.batting : null;
    winType = winner === chasingKey ? 'chase' : 'defend';
  }

  game.winner = winner;
  game.winType = winType;
  game.status = 'finished';
  await game.save();

  const user = await User.findById(game.player1.userId);
  if (user) {
    user.stats.gamesPlayed += 1;
    if (winner === 'player1') user.stats.gamesWon += 1;
    user.stats.totalRuns += player1Runs;
    user.stats.wickets += game.player2.wickets;
    await user.save();

    await Leaderboard.findOneAndUpdate(
      { userId: user._id },
      { userName: user.name, totalGames: user.stats.gamesPlayed, ... },
      { upsert: true, new: true }
    );
  }
  return { winner, winType, player1Runs, player2Runs };
}
```

Code	Explanation
<code>const chasingKey = game.currentInning === 2 ? game.batting : null;</code>	"Chasing" only makes sense in the second innings (the side batting second is chasing a target). This determines whether the winner won by successfully chasing down the target, or by defending a total set in innings 1.
<code>user.stats.wickets += game.player2.wickets;</code>	Tracks how many times the <i>human</i> managed to dismiss the bot — i.e. the human's bowling performance across the game, stored on the User document.
<code>Leaderboard.findOneAndUpdate({userId}, {...}, {upsert:true, new:true})</code>	<code>upsert: true</code> means "update if a document matching the filter exists, otherwise insert a new one" — avoids a separate find-then-create step. <code>new: true</code> returns the post-update document rather than the pre-update one (though the return value isn't used here).

11b. POST /game/create

```
router.post('/create', auth, async (req, res) => {
  const { difficulty } = req.body;
  const user = await User.findById(req.userId);
  const gameId = GameLogic.generateGameId();
  const gameData = {
    gameId,
    player1: { userId: user._id, name: user.name },
    player2: { name: 'Bot', difficulty: difficulty || 'medium', isBot: true },
    status: 'toss',
  };
  const game = new Game(gameData);
  await game.save();
  res.status(201).json({ message: 'Game created successfully', game: gameData });
});
```

Code	Explanation
<code>const user = await User.findById(req.userId);</code>	<code>req.userId</code> was attached by the auth middleware — this route never has to parse the JWT itself, it just trusts the middleware already did.
<code>status: 'toss'</code>	Every new game starts life needing a toss before anything else can happen — this is the very first state in the state machine defined by the Game schema's status enum.

11c. POST /game/toss

```

router.post('/toss', auth, async (req, res) => {
  const { gameId, guess, number } = req.body;
  const game = await Game.findOne({ gameId });
  const markov = new MarkovModel(game.player2.difficulty);
  const botNumber = markov.getRandomNumber();
  const tossResult = GameLogic.resolveToss(guess, number, botNumber);
  game.tossWinner = tossResult.winner;

  if (tossResult.winner === 'player1') {
    game.status = 'toss-choice';
    await game.save();
    return res.json({ message: 'You won the toss! Choose to bat or bowl', needsChoice: true, ...
  });
}

const botChoosesBat = Math.random() < 0.5;
game.batting = botChoosesBat ? 'player2' : 'player1';
game.bowling = botChoosesBat ? 'player1' : 'player2';
game.status = 'inning1';
await game.save();
res.json({ message: `Bot won the toss and chose to ${botChoosesBat ? 'bat' : 'bowl'}`,
needsChoice: false, ... });
});

```

Code	Explanation
<pre>const botNumber = markov.getRandomNumber();</pre>	The toss uses a purely random bot number, not a prediction — there's no player history yet to base a prediction on at this point in a fresh game.
<pre>if (tossResult.winner === 'player1') { status='toss-choice'; return; }</pre>	Branches the flow: if the human wins, the function returns early with <code>needsChoice: true</code> and waits for a separate <code>/choose</code> request. Execution never reaches the bot's auto-decision code below in this case.
<pre>const botChoosesBat = Math.random() < 0.5;</pre>	If the bot wins the toss instead, it needs to decide bat-or-bowl itself with no human input — a simple 50/50 coin flip.

11d. POST /game/choose

```

router.post('/choose', auth, async (req, res) => {
  const { gameId, choice } = req.body;
  const game = await Game.findOne({ gameId });
  if (game.status !== 'toss-choice' || game.tossWinner !== 'player1') {
    return res.status(400).json({ message: 'No pending toss choice for this game' });
  }
  game.batting = choice === 'bat' ? 'player1' : 'player2';
  game.bowling = choice === 'bat' ? 'player2' : 'player1';
  game.status = 'inning1';
  await game.save();
  res.json({ message: `You chose to ${choice} first`, batting: game.batting, bowling: game.bowling
});
});

```

Code	Explanation
<pre>if (game.status !== 'toss-choice' game.tossWinner !== 'player1')</pre>	A guard clause preventing this endpoint from being called out of sequence — e.g. calling it on a game where the bot already won the toss (and thus already decided bat/bowl automatically).

11e. POST /game/play — the per-ball engine

```
router.post('/play', auth, async (req, res) => {
  const { gameId, number } = req.body;
  const game = await Game.findOne({ gameId });
  const markov = new MarkovModel(game.player2.difficulty);
  const botRole = game.batting === 'player2' ? 'bat' : 'bowl';
  const botNumber = markov.chooseBotNumber(game.player1.numberHistory, botRole);

  game.player1.lastNumber = number;
  game.player1.numberHistory.push(number);
  game.player2.lastNumber = botNumber;

  const battingKey = game.batting;
  const bowlingKey = game.bowling;
  const battingNumber = battingKey === 'player1' ? number : botNumber;
  const bowlingNumber = bowlingKey === 'player1' ? number : botNumber;
  const ball = GameLogic.playBall(battingNumber, bowlingNumber);

  if (ball.outcome === 'wicket') game[battingKey].wickets += 1;
  else game[battingKey].runs += ball.runs;

  let finalResult = null;
  const battingAllOut = game[battingKey].wickets >= 6;

  if (game.currentInning === 1) {
    if (battingAllOut) {
      game.batting = bowlingKey;
      game.bowling = battingKey;
      game.currentInning = 2;
      game.target = game[battingKey].runs + 1;
      game.status = 'inning2';
    }
    await game.save();
  } else {
    const targetReached = game[battingKey].runs >= game.target;
    if (battingAllOut || targetReached) {
      finalResult = await finalizeGame(game);
    } else {
      await game.save();
    }
  }
}

res.json({ outcome: ball.outcome, yourNumber: number, botNumber, ...
  player1Runs: game.player1.runs, player1Wickets: game.player1.wickets,
  player2Runs: game.player2.runs, player2Wickets: game.player2.wickets,
  currentInning: game.currentInning, target: game.target, status: game.status, finalResult });
});
```

Code	Explanation
<code>const botRole = game.batting === 'player2' ? 'bat' : 'bowl';</code>	Tells the AI which strategy to use this ball — match the human's predicted number (bowling) or dodge it (batting). This is derived fresh every request rather than stored, since it flips automatically when the innings switches.
<code>game.player1.numberHistory.push(number);</code>	Every real ball played (not toss numbers) gets appended to the history array that feeds the Markov model — so the bot gets progressively better-informed as the match goes on.
<code>const battingKey = game.batting; const bowlingKey = game.bowling;</code>	These are string values 'player1'/'player2' used as <i>dynamic object keys</i> below (<code>game[battingKey]</code>). This is what lets one code path correctly handle both "human batting, bot bowling" and "bot batting, human bowling" without an if/else duplicating the whole block.
<code>game[battingKey].wickets += 1;</code>	Bracket notation for dynamic property access — equivalent to <code>game.player1.wickets += 1</code> or <code>game.player2.wickets += 1</code> depending on the runtime value of <code>battingKey</code> .
<code>if (game.currentInning === 1) { if (battingAllOut) {...swap...} }</code>	In innings 1, all-out just triggers a role swap and moves to innings 2 — the game keeps going. The target is set to <i>current batting side's runs + 1</i> (the minimum score the chasing side needs to beat it).
<code>} else { const targetReached = ...; if (battingAllOut targetReached) finalizeGame(game); }</code>	In innings 2, the same two conditions (all-out OR target reached) instead <i>end the whole match</i> , because there's no third innings — this branch calls the shared <code>finalizeGame</code> helper instead of just swapping roles.
<code>finalResult = await finalizeGame(game);</code>	When the game auto-finishes mid-/play call, the response includes the final result inline (<code>finalResult</code>) so the frontend doesn't need a second request to <code>/finish</code> to learn who won.

11f. POST /game/finish & GET /game/:gameId

```
router.post('/finish', auth, async (req, res) => {
  const { gameId } = req.body;
  const game = await Game.findOne({ gameId });
  if (game.status === 'finished') {
    return res.json({ message: 'Game already finished', winner: game.winner, ... });
  }
  const result = await finalizeGame(game);
  res.json({ message: 'Game finished', ...result });
});

router.get('/:gameId', auth, async (req, res) => {
  const game = await Game.findOne({ gameId: req.params.gameId })
    .populate('player1.userId', 'name email')
    .populate('player2.userId', 'name email');
  res.json({ game });
});
```

Code	Explanation
<code>POST /finish</code>	Lets the player concede/end a game early (an "End Game Now" button), reusing the exact same <code>finalizeGame</code> helper the automatic innings-2 completion uses — one source of truth for "what happens when a game ends."
<code>if (game.status === 'finished') { return res.json({...}); }</code>	Idempotency guard — calling <code>/finish</code> twice on an already-finished game just re-reports the existing result instead of double-counting stats by running <code>finalizeGame</code> a second time.
<code>.populate('player1.userId', 'name email')</code>	Mongoose's <code>populate</code> replaces the stored <code>ObjectId</code> reference with the actual referenced <code>User</code> document (limited to just the <code>name</code> and <code>email</code> fields here) — this is the payoff of the <code>ref: 'User'</code> declared back in the schema.
<code>req.params.gameId</code>	Express extracts the <code>:gameId</code> segment from the URL path (e.g. <code>/api/game/game_12345</code>) into <code>req.params.gameId</code> .

12. server/routes/leaderboard.js — Ranking Endpoints

server/routes/leaderboard.js

Read-only endpoints for the leaderboard and profile screens — no game logic here, just queries and rank computation.

```
router.get('/top', async (req, res) => {
  const limit = req.query.limit || 10;
  const leaderboard = await Leaderboard.find()
    .sort({ winRate: -1, totalWins: -1 })
    .limit(parseInt(limit));
  const rankedLeaderboard = leaderboard.map((entry, index) => ({
    ...entry.toObject(), rank: index + 1,
  }));
  res.json({ leaderboard: rankedLeaderboard });
});

router.get('/my-stats', auth, async (req, res) => {
  const user = await User.findById(req.userId);
  const leaderboardEntry = await Leaderboard.findOne({ userId: req.userId });
  const rank = leaderboardEntry
    ? await Leaderboard.countDocuments({ winRate: { $gt: leaderboardEntry.winRate } }) + 1
    : 0;
  res.json({ stats: { ...user.stats, rank, winRate: ... } });
});
```

Code	Explanation
<code>.sort({ winRate: -1, totalWins: -1 })</code>	-1 means descending order. Primary sort by win rate, with total wins as a tiebreaker for players sitting at the same win rate.
<code>.map((entry, index) => ({ ...entry.toObject(), rank: index + 1 })))</code>	Since the query is already sorted, array position directly gives rank. <code>.toObject()</code> converts a Mongoose document into a plain JS object so the spread operator <code>...</code> works cleanly; <code>index + 1</code> converts a zero-based array index into a human-friendly 1-based rank.
<code>Leaderboard.countDocuments({ winRate: { \$gt: leaderboardEntry.winRate } }) + 1</code>	Computes a specific user's rank without needing the full sorted list — counts how many players have a strictly higher win rate, then adds 1. If zero players beat them, they're rank 1.

13. client/src/main.jsx — Frontend Entry Point

client/src/main.jsx

The first line of frontend code the browser executes. Mounts the React component tree into the actual DOM.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Code	Explanation
<code>createRoot(document.getElementById('root'))</code>	React 19's modern mounting API — finds the empty <code><div id="root"></code> in <code>index.html</code> and creates a React "root" attached to it, where the component tree will be rendered.
<code><StrictMode><App /></StrictMode></code>	<code>StrictMode</code> is a development-only wrapper that intentionally double-invokes certain functions to help surface bugs (like impure render logic) early — it has no effect in a production build.

14. client/src/App.jsx — Routing & Auth Guards

client/src/App.jsx

Defines every URL the app responds to, and wraps each screen in a guard component that redirects based on login state.

```
function ProtectedRoute({ children }) {
  const { user, loading } = useContext(AuthContext);
  if (loading) return <div>Loading...</div>;
  if (!user) return <Navigate to="/login" />;
  return children;
}

function PublicRoute({ children }) {
  const { user, loading } = useContext(AuthContext);
  if (loading) return <div>Loading...</div>;
  if (user) return <Navigate to="/game" />;
  return children;
}

function AppRoutes() {
  return (
    <Routes>
      <Route path="/login" element={<PublicRoute><Login /></PublicRoute>} />
      <Route path="/signup" element={<PublicRoute><Signup /></PublicRoute>} />
      <Route path="/game" element={<ProtectedRoute><Game /></ProtectedRoute>} />
      <Route path="/leaderboard" element={<ProtectedRoute><Leaderboard /></ProtectedRoute>} />
      <Route path="/profile" element={<ProtectedRoute><Profile /></ProtectedRoute>} />
      <Route path="/" element={<Navigate to="/game" />} />
    </Routes>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <Router>
        <AppRoutes />
      </Router>
    </AuthProvider>
  );
}
```

Code	Explanation
<pre>const { user, loading } = useContext(AuthContext);</pre>	Reads shared authentication state from React Context (file #15) — any component anywhere in the tree can do this without props being manually passed down through every intermediate component.
<pre>if (loading) return <div>Loading... </div>;</pre>	On first page load, the app doesn't yet know if a stored token is valid — loading stays true while that check happens, preventing a flash of the wrong screen (e.g. briefly showing the login page before redirecting a still-logged-in user to /game).
<pre>ProtectedRoute: if (!user) return <Navigate to="/login" />;</pre>	<code><Navigate></code> is React Router's declarative redirect — rendering it causes an immediate client-side redirect, no full page reload.
<pre>PublicRoute: if (user) return <Navigate to="/game" />;</pre>	The inverse guard — stops an already-logged-in user from seeing the login/signup forms again if they manually navigate to those URLs.
<pre><AuthProvider><Router><AppRoutes /> </Router></AuthProvider></pre>	Nesting order matters: <code>AuthProvider</code> wraps <code>Router</code> so that every routed screen (and the guards themselves) can access <code>AuthContext</code> .

15. client/src/context/AuthContext.jsx — Global Auth State

client/src/context/AuthContext.jsx

A single source of truth for "who is logged in," shared across the whole app via React Context instead of prop-drilling.

```

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) fetchUser();
    else setLoading(false);
  }, []);

  const fetchUser = async () => {
    try {
      const response = await authAPI.getMe();
      setUser(response.data.user);
    } catch (err) {
      localStorage.removeItem('token');
      setUser(null);
    } finally {
      setLoading(false);
    }
  };

  const signup = async (name, email, password, confirmPassword) => {
    const response = await authAPI.signup({ name, email, password, confirmPassword });
    return response.data; // deliberately does NOT store token / setUser
  };

  const login = async (email, password) => {
    const response = await authAPI.login({ email, password });
    localStorage.setItem('token', response.data.token);
    setUser(response.data.user);
    return response.data;
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, loading, error, signup, login, logout, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};

```

Code	Explanation
<code>createContext()</code>	Creates a "channel" that any descendant component can subscribe to with <code>useContext(AuthContext)</code> , without every intermediate component needing to pass the data down manually as props.
<code>useEffect(() => {...}, [])</code>	The empty dependency array <code>[]</code> means this runs exactly once, right after the component first mounts — used here to check for a saved login on page load/refresh.
<code>if (token) fetchUser(); else setLoading(false);</code>	If a token exists in <code>localStorage</code> from a previous session, verify it's still valid by asking the server who it belongs to. If there's no token at all, skip straight to "not loading" since there's nothing to check.
<code>catch (err) { localStorage.removeItem('token'); setUser(null); }</code>	If the stored token is expired or invalid, the <code>/auth/me</code> request fails — this cleans up the stale token so the app doesn't keep retrying with dead credentials.
<code>signup: ...return response.data; // no token storage</code>	Intentional design choice: <code>signup</code> creates the account but does not log the user in automatically. The token from the response is discarded — the user is instead routed to the login screen to authenticate for real.
<code>login: localStorage.setItem('token', ...); setUser(...);</code>	This is where an actual session begins — the JWT is persisted so it survives a page refresh, and user state updates so every component re-renders reflecting the logged-in state.

16. client/src/api/api.js — Centralized API Client

client/src/api/api.js

The single place in the frontend that knows the backend's base URL and endpoint paths. Every component calls functions from here instead of using axios directly.

```
const api = axios.create({ baseURL: 'http://localhost:5000/api' });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export const authAPI = {
  signup: (data) => api.post('/auth/signup', data),
  login: (data) => api.post('/auth/login', data),
  getMe: () => api.get('/auth/me'),
};

export const gameAPI = {
  createGame: (difficulty) => api.post('/game/create', { difficulty }),
  playToss: (gameId, guess, number) => api.post('/game/toss', { gameId, guess, number }),
  chooseBatBowl: (gameId, choice) => api.post('/game/choose', { gameId, choice }),
  playBall: (gameId, number) => api.post('/game/play', { gameId, number }),
  finishGame: (gameId) => api.post('/game/finish', { gameId }),
  getGame: (gameId) => api.get(`/game/${gameId}`),
};

export const leaderboardAPI = {
  getTop: (limit = 10) => api.get(`/leaderboard/top?limit=${limit}`),
  getUserStats: () => api.get('/leaderboard/my-stats'),
};
```

Code	Explanation
<code>axios.create({ baseURL: ... })</code>	Creates a pre-configured axios instance — every call through <code>api</code> automatically prefixes the base URL, so route functions below only need to specify the path (<code>/auth/login</code> , not the full URL).
<code>api.interceptors.request.use((config) => {...})</code>	An axios "interceptor" runs on <i>every single outgoing request</i> made through this instance, before it's sent. This is what automatically attaches the JWT to every API call — no individual component ever has to remember to add the auth header manually.
<code>export const gameAPI = { createGame: (difficulty) => ... }</code>	Each property is a small wrapper function that shapes the request body/URL correctly for that specific endpoint — components call e.g. <code>gameAPI.playBall(gameId, 4)</code> instead of remembering the exact POST body shape themselves.

17. client/src/components/Auth/Login.jsx & Signup.jsx

client/src/components/Auth/{Login,Signup}.jsx

The two authentication forms. Signup creates an account then hands off to Login; Login establishes the actual session.

```
// Signup.jsx (relevant excerpt)
const handleSubmit = async (e) => {
  e.preventDefault();
  if (password !== confirmPassword) { setError('Passwords do not match'); return; }
  setLoading(true);
  try {
    await signup(name, email, password, confirmPassword);
    navigate('/login', { state: { justSignedUp: true, email } });
  } catch (err) {
    setError(err.response?.data?.message || 'Signup failed');
  } finally {
    setLoading(false);
  }
};

// Login.jsx (relevant excerpt)
const location = useLocation();
const [email, setEmail] = useState(location.state?.email || '');
const [justSignedUp, setJustSignedUp] = useState(Boolean(location.state?.justSignedUp));

const handleSubmit = async (e) => {
  e.preventDefault();
  setLoading(true);
  try {
    await login(email, password);
    navigate('/game');
  } catch (err) {
    setError(err.response?.data?.message || 'Login failed');
  } finally {
    setLoading(false);
  }
};
```

Code	Explanation
<code>e.preventDefault();</code>	Stops the browser's native form submission (which would trigger a full page reload) so React can handle the submit entirely via JavaScript/fetch instead.
<code>navigate('/login', { state: { justSignedUp: true, email } });</code>	React Router lets you attach arbitrary state to a navigation, retrievable on the destination page via <code>useLocation()</code> — used here to pass the just-registered email and a flag across the redirect without a URL query string.
<code>useState(location.state?.email '')</code>	Reads that passed-along state as the <i>initial</i> value of the email input — pre-fills the login form so the user doesn't have to retype what they just entered on signup.
<code>err.response?.data?.message 'Login failed'</code>	Optional chaining safely drills into the axios error object (which might not have a <code>.response</code> at all if the network request failed outright), falling back to a generic message if the server didn't send a specific one.

18. client/src/components/Game/Game.jsx — Pre-Match Lobby

client/src/components/Game/Game.jsx

The screen shown right after login — lets the player pick a bot difficulty and creates the Game document, then swaps to the live gameplay component.

```
const [gameState, setGameState] = useState(null);
const [difficulty, setDifficulty] = useState('medium');
const [gameStarted, setGameStarted] = useState(false);

const handleCreateGame = async () => {
  setLoading(true);
  try {
    const response = await gameAPI.createGame(difficulty);
    setGameState(response.data.game);
    setGameStarted(true);
  } catch (err) {
    setError(err.response?.data?.message || 'Failed to create game');
  } finally {
    setLoading(false);
  }
};

return (
  ...
  {!gameStarted ? (
    <div className="game-setup">...difficulty radio buttons...</div>
  ) : (
    <GameBoard game={gameState} user={user} />
  )}
);
```

Code	Explanation
<pre>const [gameStarted, setGameStarted] = useState(false);</pre>	A simple boolean flag that toggles which JSX block renders — this is the conditional-rendering pattern that swaps the difficulty picker for the actual <GameBoard> component once a game exists.
<pre>{!gameStarted ? (...) : (<GameBoard game= {gameState} user={user} />)}</pre>	Standard React ternary-in-JSX conditional rendering. Once gameStarted flips true, gameState (the object returned from /game/create) is passed down as a prop so GameBoard knows the gameId to send with every subsequent API call.

19. client/src/components/Game/GameBoard.jsx — Live Gameplay

client/src/components/Game/GameBoard.jsx

The most stateful component in the app — a client-side state machine (gamePhase) that mirrors the backend's status field, driving the toss, the optional bat/bowl choice, ball-by-ball play, and the final result screen.

```
const [gamePhase, setGamePhase] = useState('toss');
const [battingKey, setBattingKey] = useState(null);
const [bowlingKey, setBowlingKey] = useState(null);
const [currentInning, setCurrentInning] = useState(1);
const [scores, setScores] = useState({ player1Runs: 0, player1Wickets: 0, player2Runs: 0,
player2Wickets: 0 });

const handleToss = async () => {
  const response = await gameAPI.playToss(game.gameId, guess, selectedNumber);
  const data = response.data;
  if (data.needsChoice) {
    setGamePhase('toss-choice');
  } else {
    setBattingKey(data.batting);
    setBowlingKey(data.bowling);
    setGamePhase('playing');
  }
};

const handlePlayBall = async () => {
  const response = await gameAPI.playBall(game.gameId, selectedNumber);
  const data = response.data;
  setScores({ player1Runs: data.player1Runs, player1Wickets: data.player1Wickets,
    player2Runs: data.player2Runs, player2Wickets: data.player2Wickets });

  if (data.status === 'finished') {
    setFinalResult(data.finalResult);
    setGamePhase('finished');
  } else if (data.currentInning !== currentInning) {
    setCurrentInning(data.currentInning);
    setBattingKey((prev) => (prev === 'player1' ? 'player2' : 'player1'));
    setBowlingKey((prev) => (prev === 'player1' ? 'player2' : 'player1'));
  }
};
```

Code	Explanation
<pre>const [gamePhase, setGamePhase] = useState('toss');</pre>	A local mirror of the backend's <code>status</code> field, driving which JSX block is shown: <code>'toss' → 'toss-choice' → 'playing' → 'finished'</code> . Kept as separate frontend state (rather than reading straight from a server response every render) so the UI can transition instantly on each API response.
<pre>if (data.needsChoice) setGamePhase('toss- choice'); else {...set roles...; setGamePhase('playing');}</pre>	Mirrors the backend's branch in <code>routes/game.js</code> 's <code>/toss</code> handler: if the human won the toss, the UI needs to show the bat/bowl choice buttons next; if the bot won, the server already decided everything, so the UI can jump straight into gameplay with the roles it returned.
<pre>setScores({ player1Runs: data.player1Runs, ... })</pre>	The scoreboard is fully server-authoritative — after every ball, the frontend just overwrites its local score state with whatever the backend computed, rather than trying to calculate scores independently on the client (which would be both duplicated logic and a cheating vector).
<pre>else if (data.currentInning !== currentInning) { ...flip battingKey/bowlingKey... }</pre>	Detects an innings transition by comparing the inning number just returned against what the component currently has stored. Since the game is always exactly two innings, "the inning changed" always means "batting and bowling roles just swapped" — so the local state is flipped to match without needing the server to explicitly resend which key is which.
<pre>setBattingKey((prev) => (prev === 'player1' ? 'player2' : 'player1'))</pre>	The <i>functional updater</i> form of <code>setState</code> — instead of passing a fixed new value, you pass a function that receives the current state and returns the next one. This avoids stale-closure bugs when the update depends on the previous value.

20. Leaderboard.jsx & Profile.jsx — Read-Only Stat Screens

client/src/components/Leaderboard/Leaderboard.jsx & Profile/Profile.jsx

Both follow the same simple pattern: fetch data once when the component mounts, then render it. No state machine, no mutations.

```
// Leaderboard.jsx
useEffect(() => {
  if (!user) navigate('/login');
  else fetchLeaderboard();
}, [user, navigate]);

const fetchLeaderboard = async () => {
  const response = await leaderboardAPI.getTop(20);
  setLeaderboard(response.data.leaderboard);
};

// rendered as a <table>, one <tr> per entry:
{leaderboard.map((entry, index) => (
  <tr key={entry._id}>
    <td>{index + 1 === 1 ? '🏆' : index + 1}</td>
    <td>{entry.userName}</td>
    <td>{entry.winRate}%</td>
  </tr>
))}
```

Code	Explanation
<code>useEffect(() => {...}, [user, navigate])</code>	Runs whenever user or navigate changes (in practice, mainly on first mount once user becomes available from context). Doubles as an auth guard at the component level, on top of the route-level <code>ProtectedRoute</code> .
<code>leaderboard.map((entry, index) => (<tr key={entry._id}>...</tr>))</code>	Standard React list rendering — <code>.map()</code> transforms an array of data into an array of JSX elements. The <code>key</code> prop (using MongoDB's <code>_id</code>) helps React efficiently track which row is which across re-renders.
<code>index + 1 === 1 ? '🏆' : index + 1</code>	Since the array arrives already sorted by rank, array position doubles as the displayed rank — with a special-cased medal emoji for the top spot.

21. End-to-End Request Lifecycle

Tracing one complete user journey through every file above, in the order things actually execute.

- **Page load:** `main.jsx` mounts `App.jsx` → `AuthProvider` checks `localStorage` for a token → no token found → loading becomes false with user still null.
- **Signup:** `Signup.jsx` submit → `AuthContext.signup()` → `api.js` → `POST /api/auth/signup` → `routes/auth.js` → `new User(...).save()` triggers the `bcrypt pre('save')` hook in `models/User.js` → JWT generated but discarded client-side → `navigate('/login', {state: {justSignedUp:true, email}})`.
- **Login:** `Login.jsx` shows the green success banner from that navigation state → submit → `AuthContext.login()` → `POST /api/auth/login` → `user.comparePassword()` checked → token saved to `localStorage`, user state set → `PublicRoute` guard now redirects away from `/login` automatically → `navigate('/game')`.
- **Lobby:** `Game.jsx` → pick difficulty → `POST /game/create` → `routes/game.js` inserts a Game document with `status: 'toss'` → `<GameBoard>` mounts.
- **Toss:** `GameBoard.jsx` → guess odd/even + number → `POST /game/toss` → `GameLogic.resolveToss()` decides the winner deterministically from the sum's parity → either waits for `/game/choose` (human won) or auto-decides bat/bowl (bot won) → `status: 'inning1'`.
- **Gameplay loop:** each number tap → `POST /game/play` → `MarkovModel.chooseBotNumber()` generates the bot's move from the human's `numberHistory` → `GameLogic.playBall()` resolves runs/wicket → dynamic `game[battingKey]` updates → `innings-switch` or `auto-finish` logic checked on every single call.
- **Finish:** `finalizeGame()` writes the winner, updates `User.stats`, upserts the Leaderboard entry → `GameBoard.jsx` shows the result screen.
- **Stats:** visiting `/leaderboard` or `/profile` later reflects those exact updated numbers via `GET /leaderboard/top` and `GET /leaderboard/my-stats`.